



Prueba Técnica

Perfil Fullstack (.NET Core 9 + Angular 18–20)

Objetivo: Evaluar tu dominio en desarrollo fullstack utilizando tecnologías modernas, buenas prácticas de arquitectura, diseño, patrones de software y capacidades de implementación de API + Frontend.



Descripción General de la Prueba

Deberás desarrollar un **módulo completo de catálogo (CRUD/ABC)** utilizando **.NET Core 9** y **Angular (18–20)**, aplicando arquitectura limpia (Clean Architecture) y/o Vertical Slice Architecture.

El módulo debe incluir:

- Administración completa de un catálogo (ej. *Productos, Categorías, Cursos, Tiendas, Proveedores*, etc.)
- Altas, Bajas, Cambios (ABC)
- Listado con filtros y paginación (deseable)
- Formularios con validaciones
- Vista de detalle (deseable)
- Documentación del proyecto

La solución debe estar dividida en backend (API) y frontend (cliente web), siguiendo buenas prácticas modernas.



Requerimientos Técnicos



Arquitectura

La solución debe implementar alguno de estos estilos:

- **Clean Architecture**
- **Vertical Slice Architecture** (deseable)

Se evaluará:

- Separación clara de capas (Api / Application / Domain / Infrastructure)
- Uso de CQRS (si aplica), DTOs, Profiles/Mapping
- Manejo de reglas de dominio y validaciones consistentes
- Versionamiento de API, filtros y paginación (deseable)

Backend (.NET Core 9 – C#)

Requerido:

- Implementar **CRUD completo** del catálogo utilizando:
 - **EF Core** con **SQLite** o
 - **Microsoft.EntityFrameworkCore.InMemory**
- Manejo correcto de:
 - Respuestas HTTP (200/201/400/404/500)
 - Excepciones globales
 - Logs básicos
- Aplicar:
 - Principios SOLID
 - Patrones de diseño (Repository, Strategy, Factory, etc. cuando apliquen)

Deseable:

- Seguridad (JWT + Políticas básicas)
- Configuración de CORS
- Unit tests en backend (xUnit / NUnit / MSTest)

Frontend (Angular 18–20)

Requerido:

- Implementar una SPA con Angular y:
 - Tabla de catálogo con filtros y paginación (deseable)
 - Formularios reactivos (Reactive Forms) con validaciones
 - CRUD completo consumiendo la API
 - Guards / Interceptors
 - Manejo de estado básico (Signals o RxJS: Subjects/BehaviorSubjects)

Deseable:

- Lazy Loading en módulos o rutas
- Unit Tests (Karma/Jasmine o Jest)
- RxJS aplicado correctamente
- Mejoras UI/UX básicas

Documentación – Entrega Final

Debe incluir al menos:

- **README** con:
 - Cómo correr backend y frontend
 - Requerimientos previos

- Tecnología utilizada
- Arquitectura seleccionada y razones
- Decisiones técnicas clave
- **Colección de Postman** (deseable)
 - Para probar las operaciones ABC
 - Con ejemplos de request/response



Entrega del Proyecto

Subir la solución completa a un repositorio público (o privado con acceso) en:

- GitHub
- GitLab
- Bitbucket
- Azure DevOps repos

Incluye en tu respuesta:

- ✓ **URL del repositorio**
- ✓ Instrucciones para correr el proyecto



Criterios de Evaluación

El proyecto se evaluará considerando:

1. Arquitectura y buenas prácticas

Estructura, separación de capas, uso de patrones, SOLID, CQRS y diseño modular.

2. Calidad del código

Claridad, consistencia, nombres, manejo de errores, uso adecuado del framework.

3. Habilidad Fullstack real

Conexión API ↔ Frontend, validaciones, estado, modularidad, seguridad básica.

4. Pruebas

Aunque son deseables, suman significativamente si se incluyen.

5. Documentación

Claridad, orden y explicación de decisiones.